Image Classification using CNN

Last Updated : 5 Aug, 2025

Image classification is a key task in machine learning where the goal is to assign a label to an image based on its content. Convolutional Neural Networks (CNNs) are specifically designed to analyze and interpret images. Unlike traditional neural networks, they are good at detecting patterns, shapes and textures by breaking down an image into smaller parts and learning from these details. By processing these patterns in multiple layers, it can identify increasingly complex features making them effective for tasks like classifying images of animals, objects or scenes. In this article, we will see how CNNs work and how to use them to build an image classifier.

Key Components of CNNs

A Convolutional Neural Network (CNN) is made up of several layers, each designed to perform a specific function in processing images:

1. **Convolutional Layers:** Filters or kernels that detect features such as edges or textures.
2. **ReLU Activation:** Adds non-linearity, helping the model learn complex patterns.
3. **Pooling Layers:** Reduce the dimensions of the image making the network more efficient while preserving important features.
4. **Fully Connected Layers:** After feature extraction, these layers make the final prediction based on the detected patterns.
5. **Softmax Output:** Converts the network's output into probabilities, showing the likelihood of each class.

How CNNs Work for Image Classification?

The process of image classification with a CNN involves several stages:

1. **Preprocessing the Image:** Images need to be preprocessed before feeding them into the CNN. This includes resizing, normalizing and sometimes augmenting images to make the model more robust and reduce overfitting.
2. **Feature Extraction:** CNNs automatically detect features from images, starting with simple features like edges and progressing to more complex patterns like objects or faces as we go deeper into the network.
3. **Classification:** After extracting features, the fully connected layers use the learned information to classify the image. Based on the features detected, the model assigns the image to one of the predefined categories.

Implementation of Image Classification using CNN

Lets see the implementation of Image Classification step-by-step:

Step 1: Importing Libraries

We will be using Tensorflow and Matplotlib libraries for building, training and visualizing training and validation accuracy of the model.

```
import tensorflow as tf
from tensorflow.keras import layers, models, datasets
import matplotlib.pyplot as plt
```

Step 2: Downloading and Preparing the Dataset

Next we load the CIFAR-10 dataset and preprocess it. It consists of 60,000 32x32 color images across 10 categories.

- **Scaling:** We scale the image pixel values from [0, 255] to [0, 1] by dividing by 255.
- **One-hot encoding:** Converts the labels (0-9) into a one-hot vector (e.g., for label 2: [0, 0, 1, 0, 0, 0, 0, 0, 0, 0]).

```
(x_train, y_train), (x_test, y_test) = datasets.cifar10.load_data()

x_train, x_test = x_train / 255.0, x_test / 255.0

num_classes = 10
y_train = tf.keras.utils.to_categorical(y_train, num_classes)
y_test  = tf.keras.utils.to_categorical(y_test , num_classes)
```

Output:

```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 3s 0us/step
```

Downloading the Dataset

Step 3: Building the CNN Model

Now, we define the CNN architecture and start with convolutional layers followed by max-pooling layers, flatten the output and then feed it into fully connected layers.

- **Flatten Layer:** Converts the 2D matrix into a 1D vector for the dense layers.
- **Dense Layers:** Fully connected layers used for decision making, with the final layer using softmax activation to predict probabilities.

```
model = models.Sequential([

    layers.Conv2D(32, (3,3), activation='relu', padding='same', input_shape=(32,32,3)),
    layers.MaxPooling2D(2,2),

    layers.Conv2D(64, (3,3), activation='relu', padding='same'),
    layers.MaxPooling2D(2,2),

    layers.Conv2D(64, (3,3), activation='relu', padding='same'),
    layers.Flatten(),

    layers.Dense(64, activation='relu'),
    layers.Dense(num_classes, activation='softmax')
```

```
])

model.summary()
```

Output:

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 32, 32, 32)	896
max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)	0
conv2d_1 (Conv2D)	(None, 16, 16, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 64)	0
conv2d_2 (Conv2D)	(None, 8, 8, 64)	36,928
flatten (Flatten)	(None, 4096)	0
dense (Dense)	(None, 64)	262,208
dense_1 (Dense)	(None, 10)	650

Total params: 319,178 (1.22 MB)
Trainable params: 319,178 (1.22 MB)
Non-trainable params: 0 (0.00 B)

Building the CNN Model

Step 4: Compiling and Training the Model

We then compile the model by defining the optimizer, loss function and evaluation metric, followed by training. Adam optimizer is used as it adjusts the learning rate during training.

```
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

history = model.fit(x_train, y_train,
                    epochs=15,
                    batch_size=64,
                    validation_split=0.2,
                    verbose=2)
```

Output:

```
Epoch 1/15
625/625 - 79s - 127ms/step - accuracy: 0.4285 - loss: 1.5590 - val_accuracy: 0.5326 - val_loss: 1.3287
Epoch 2/15
625/625 - 76s - 122ms/step - accuracy: 0.5885 - loss: 1.1545 - val_accuracy: 0.6333 - val_loss: 1.0527
Epoch 3/15
625/625 - 82s - 130ms/step - accuracy: 0.6521 - loss: 0.9879 - val_accuracy: 0.6549 - val_loss: 0.9890
Epoch 4/15
625/625 - 83s - 133ms/step - accuracy: 0.6912 - loss: 0.8735 - val_accuracy: 0.6643 - val_loss: 0.9568
Epoch 5/15
625/625 - 77s - 123ms/step - accuracy: 0.7201 - loss: 0.7993 - val_accuracy: 0.6894 - val_loss: 0.8950
Epoch 6/15
625/625 - 77s - 123ms/step - accuracy: 0.7473 - loss: 0.7260 - val_accuracy: 0.6935 - val_loss: 0.8936
Epoch 7/15
625/625 - 82s - 132ms/step - accuracy: 0.7667 - loss: 0.6624 - val_accuracy: 0.7100 - val_loss: 0.8667
Epoch 8/15
625/625 - 81s - 130ms/step - accuracy: 0.7890 - loss: 0.5989 - val_accuracy: 0.6991 - val_loss: 0.9227
Epoch 9/15
625/625 - 70s - 112ms/step - accuracy: 0.8102 - loss: 0.5452 - val_accuracy: 0.7153 - val_loss: 0.8673
Epoch 10/15
625/625 - 84s - 134ms/step - accuracy: 0.8268 - loss: 0.4942 - val_accuracy: 0.7150 - val_loss: 0.8940
Epoch 11/15
625/625 - 78s - 124ms/step - accuracy: 0.8437 - loss: 0.4402 - val_accuracy: 0.7071 - val_loss: 0.9735
Epoch 12/15
625/625 - 72s - 115ms/step - accuracy: 0.8625 - loss: 0.3868 - val_accuracy: 0.7190 - val_loss: 0.9592
Epoch 13/15
625/625 - 71s - 114ms/step - accuracy: 0.8783 - loss: 0.3439 - val_accuracy: 0.7205 - val_loss: 1.0273
Epoch 14/15
625/625 - 73s - 116ms/step - accuracy: 0.8905 - loss: 0.3079 - val_accuracy: 0.7219 - val_loss: 1.0690
Epoch 15/15
625/625 - 81s - 130ms/step - accuracy: 0.9075 - loss: 0.2625 - val_accuracy: 0.7146 - val_loss: 1.1716
Test accuracy = 0.704
```

Training the Model

Step 5: Evaluating the Model

After training, we evaluate the model on the test dataset to check how well it performs on unseen data.

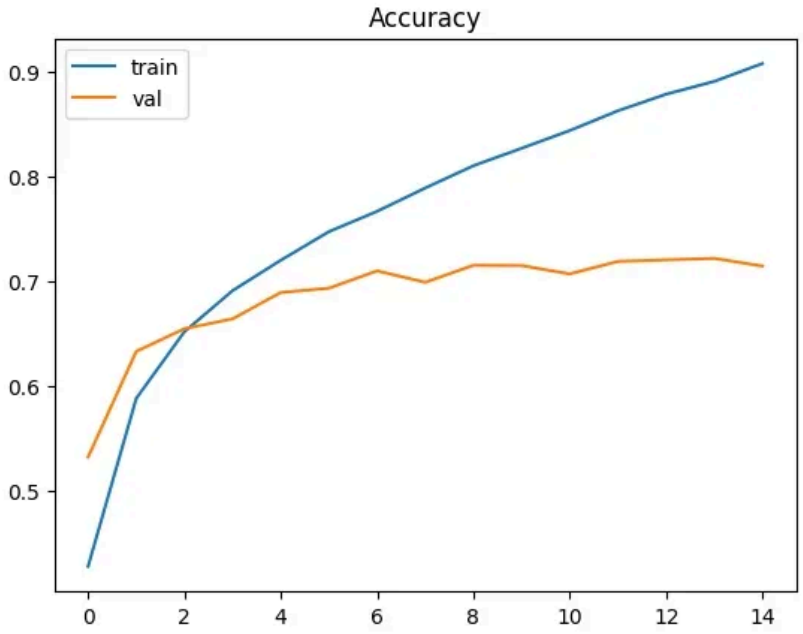
```
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)

print(f"Test accuracy = {test_acc:.3f}")
```

Step 6: Plotting of Accuracy Curves

Finally, we visualize the training and validation accuracy during training using matplotlib.

```
plt.plot(history.history['accuracy'], label='train')
plt.plot(history.history['val_accuracy'], label='val')
plt.legend()
plt.title('Accuracy')
plt.show()
```

Output:



Plotting of Accuracy

Benefits of Using CNNs for Image Classification

1. **Automatic Feature Learning:** CNNs automatically learn relevant features from images, reducing the need for manual feature extraction and making them more adaptable to different tasks.
2. **Translation Invariance:** They can recognize objects in images regardless of their position making them more robust to changes in image orientation.
3. **Efficiency:** Pooling layers help reduce the image's size making the model more computationally efficient while retaining key features.
4. **Scalability:** They can handle large datasets effectively, as they can process high volumes of data and continue improving with more examples.

Challenges in Image Classification

While CNNs have several advantages, they also come with certain challenges that we need to be solve while implementing them.

1. **Overfitting:** CNNs can overfit if the model is too complex or if there's limited data. Techniques like data augmentation and dropout help mitigate this.
2. **Computational Demands:** Training CNNs requires significant computational resources, often relying on high-performance GPUs or cloud services.
3. **Data Quality:** High-quality, labeled datasets are important for accurate classification. Poor data can lead to inaccurate predictions.
4. **Long Training Times:** Training deep CNN models can be computationally expensive and time-consuming, especially with large datasets.